BUILD YOUR OWN X

Bản đồ tự xây công nghệ

LƯU TRỮ

GIAO TIẾP

THỰC THI

Nắm tổng quan · Chọn bài vừa sức · Hiểu cơ chế bên trong

31 nhóm hướng dẫn · Chọn dự án đầu tiên · Nguồn có hyperlink
Tài liệu độc lập, đọc ngày 24/10/2026.

Mục lục có thể bấm

Bản PDF/PPTX tải về: bấm để nhảy tới phần tương ứng.
Trên Moda: liên kết mở bài đọc cùng phần trên GitHub Radar.

Repo này thực sự cung cấp gì?	tr. 3
31 nhóm công nghệ / 01	tr. 4
31 nhóm công nghệ / 02	tr. 5
Chọn bài đầu tiên theo câu hỏi	tr. 6
Một vòng học tạo ra hiểu biết	tr. 7
Dữ liệu & quản lý phiên bản	tr. 8
Web: server & browser	tr. 9
Ngôn ngữ & công cụ lập trình	tr. 10
Hệ thống & hạ tầng	tr. 11
AI, neural network & nhận dạng	tr. 12
Đồ họa, game & mô phỏng	tr. 13
Giao diện & công cụ cá nhân	tr. 14
Ví dụ thật: bước đầu làm database	tr. 15
Lộ trình gợi ý theo nền tảng	tr. 16
Tự kiểm tra: hiểu hay mới làm theo?	tr. 17
Những điều dễ hiểu nhầm	tr. 18
Phiếu bắt đầu dự án của bạn	tr. 19
Nguồn & phạm vi kiểm chứng	tr. 20

Repo này thực sự cung cấp gì?

Đọc phần này trên GitHub Radar ↗ · Tài liệu gốc ↗

build-your-own-x tập hợp hướng dẫn từng bước để tái tạo các công nghệ quen thuộc. Bạn dùng README như một mục lục: chọn thứ muốn xây, chọn ngôn ngữ, rồi đi tới tài liệu bên ngoài. Repo không phải một ứng dụng duy nhất để clone về và chạy. [S1]

Triết lý học tập

Bạn hiểu sâu hơn khi phải tự biến khái niệm thành một phiên bản có thể quan sát và thay đổi. Việc học nằm ở tài liệu đích và những quyết định khi bạn triển khai.

Cấu trúc đã kiểm tra

Tại commit nguồn được ghim, phần Tutorials có 31 nhóm, gồm cả Distributed Systems và Uncategorized. Bản đồ hai trang sau phản ánh các tiêu đề thực tế trong phần này.

Một mục hướng dẫn cho biết gì?

Tên thường đi kèm ngôn ngữ và liên kết tới bài viết, sách, video hoặc dự án. Các tài liệu có phạm vi, yêu cầu đầu vào và độ mới khác nhau.

Giới hạn của bản tổng quan

Report giải thích cách chọn và học, chưa chạy thử toàn bộ hướng dẫn. Các gợi ý đầu ra và lộ trình là đề xuất của người viết, không phải chương trình chính thức của repo.

31 nhóm công nghệ / 01

Tên nhóm mở mục tương ứng trong README. Ví dụ mở tài liệu được liệt kê; không phải xếp hạng độ khó hay chất lượng. [S1]

Distributed Systems	Hệ thống phân tán và luồng sự kiện. Hướng dẫn mẫu (Java) ↗
3D Renderer	Biến mô hình thành hình ảnh. Hướng dẫn mẫu (C++) ↗
AI Model	Các hướng xây mô hình AI. Hướng dẫn mẫu (Python) ↗
Augmented Reality	Kết hợp nội dung ảo với hình ảnh thực. Hướng dẫn mẫu (C#) ↗
BitTorrent Client	Giao tiếp tải dữ liệu theo giao thức BitTorrent. Hướng dẫn mẫu (C#) ↗
Blockchain / Cryptocurrency	Khám phá cơ chế blockchain/tiền mã hóa. Hướng dẫn mẫu (ATS) ↗
Bot	Tự động hóa tương tác hoặc tác vụ. Hướng dẫn mẫu (Haskell) ↗
Command-Line Tool	Công cụ chạy từ dòng lệnh. Hướng dẫn mẫu (Go) ↗
Database	Tổ chức, lưu và truy xuất dữ liệu. Hướng dẫn mẫu (C) ↗
Docker	Tìm hiểu một phần cơ chế container. Hướng dẫn mẫu (C) ↗
Emulator / Virtual Machine	Mô phỏng máy hoặc môi trường thực thi. Hướng dẫn mẫu (C) ↗
Front-end Framework / Library	Khám phá cơ chế framework giao diện. Hướng dẫn mẫu (JavaScript) ↗
Game	Xây trò chơi và tương tác. Hướng dẫn mẫu (C) ↗
Git	Hiểu cơ chế quản lý phiên bản. Hướng dẫn mẫu (Haskell) ↗
Memory Allocator	Tìm hiểu cách cấp phát bộ nhớ. Hướng dẫn mẫu (C) ↗
Network Stack	Khám phá các lớp giao tiếp mạng. Hướng dẫn mẫu (C) ↗

31 nhóm công nghệ / 02

Tên nhóm mở mục tương ứng trong README. Ví dụ mở tài liệu được liệt kê; không phải xếp hạng độ khó hay chất lượng. [S1]

Neural Network	Xây và quan sát mạng neural. Hướng dẫn mẫu (C#) ↗
Operating System	Tìm hiểu thành phần hệ điều hành. Hướng dẫn mẫu (Assembly) ↗
Physics Engine	Mô phỏng trạng thái và tương tác vật lý. Hướng dẫn mẫu (C) ↗
Processor	Khám phá một bộ xử lý. Hướng dẫn mẫu (Verilog) ↗
Programming Language	Đọc và thực thi ngôn ngữ lập trình. Hướng dẫn mẫu ((any)) ↗
Regex Engine	So khớp mẫu văn bản. Hướng dẫn mẫu (C) ↗
Search Engine	Tổ chức và tìm kiếm thông tin. Hướng dẫn mẫu (CSS) ↗
Shell	Nhận và xử lý dòng lệnh. Hướng dẫn mẫu (C) ↗
Template Engine	Sinh nội dung từ mẫu. Hướng dẫn mẫu (JavaScript) ↗
Text Editor	Xây công cụ soạn thảo văn bản. Hướng dẫn mẫu (C) ↗
Visual Recognition System	Xử lý bài toán nhận dạng hình ảnh. Hướng dẫn mẫu (Python) ↗
Voxel Engine	Xây môi trường dựng hình theo voxel. Hướng dẫn mẫu (C++) ↗
Web Browser	Khám phá thành phần trình duyệt. Hướng dẫn mẫu (Rust) ↗
Web Server	Nhận yêu cầu và phục vụ nội dung web. Hướng dẫn mẫu (C#) ↗
Uncategorized	Các dự án khác, đọc theo mục tiêu riêng. Hướng dẫn mẫu ((any)) ↗

Chọn bài đầu tiên theo câu hỏi

Đọc phần này trên GitHub Radar ↗ · Tài liệu gốc ↗

Đừng bắt đầu bằng công nghệ có tên lớn nhất. Hãy chọn một câu hỏi bạn đang thật sự muốn hiểu, rồi giới hạn thành một sản phẩm nhỏ. Bảng lựa chọn dưới đây là gợi ý của người viết, dựa trên các nhóm có trong README. [S1]

Tôi muốn thấy kết quả sớm

Cân nhắc Command-Line Tool, Template Engine hoặc một Web Server nhỏ bằng ngôn ngữ đã quen. Đọc yêu cầu của chính tutorial trước khi chọn.

Tôi thường dùng công cụ nhưng chưa hiểu nó

Chọn Database, Git, Shell hoặc Front-end Framework / Library tương ứng với thứ bạn đã dùng. Kinh nghiệm sử dụng giúp bạn đặt câu hỏi cụ thể hơn.

Tôi thích nền tảng hệ thống

Khám phá Docker, Network Stack, Operating System, Memory Allocator hoặc Emulator. Xác minh kiến thức hệ điều hành, công cụ và nền tảng mà tài liệu yêu cầu.

Tôi muốn học AI hoặc đồ họa

Chọn bài có phạm vi nhỏ trong Neural Network, AI Model hoặc 3D Renderer. Kiểm tra yêu cầu toán, dữ liệu, thư viện và tài nguyên tính toán ở tài liệu đích.

Một vòng học tạo ra hiểu biết

Đọc phần này trên GitHub Radar ↗ · Tài liệu gốc ↗

Vòng học đề xuất: đặt câu hỏi → chọn nguồn → xây phiên bản nhỏ → thay đổi → giải thích lại. Đích đến là vừa chạy được vừa nói rõ vì sao chương trình hoạt động, không chỉ gõ lại mã mẫu. [S1]

01 / Chốt điều muốn hiểu

Ví dụ của người viết: “Một web server đọc yêu cầu và gửi phản hồi bằng cách nào?”. Chọn một câu hỏi thay vì học mọi phần cùng lúc.

02 / Kiểm tra tutorial

Đọc mục tiêu, ngôn ngữ, công cụ, phiên bản và điểm kết thúc. Ghi điều bạn đã biết và chỗ cần bổ sung trước.

03 / Xây và kiểm tra từng phần

Làm một mốc nhỏ có đầu vào và đầu ra rõ. Lưu kết quả đúng cùng một trường hợp lỗi để hiểu giới hạn.

04 / Tự sửa rồi kể lại

Thay một hành vi nhỏ và dự đoán kết quả trước khi chạy. Sau đó vẽ đầu vào → xử lý → đầu ra và ghi phần chưa hỗ trợ.

Dữ liệu & quản lý phiên bản

Đọc phần này trên GitHub Radar ↗ · Tài liệu gốc ↗

Hai nhóm Database và Git phù hợp khi bạn muốn hiểu cách phần mềm lưu thông tin hoặc biểu diễn lịch sử thay đổi. README có nhiều cách tiếp cận và ngôn ngữ; hãy chọn phạm vi theo điều bạn muốn giải thích được. [S1]

Database: câu hỏi nên mang theo

Dữ liệu được tổ chức ở đâu? Chương trình tìm lại bản ghi như thế nào? Khi tắt và mở lại, những gì còn tồn tại? Đây là câu hỏi học tập, không phải tính năng được mọi tutorial cam kết.

Git: câu hỏi nên mang theo

Một phiên bản được biểu diễn ra sao? Các lần thay đổi liên hệ thế nào? Chọn tài liệu theo thao tác mà nó thật sự triển khai, thay vì coi bản nhỏ là Git đầy đủ.

Đầu ra gợi ý

Tự vẽ mô hình lưu trữ của phần đã làm; tạo một dữ liệu hoặc thay đổi mới và dự đoán kết quả. Ghi rõ tính năng đang thiếu.

Chuẩn bị

Ngôn ngữ lập trình quen thuộc, khả năng đọc/ghi tệp theo yêu cầu bài và cách quan sát lỗi. Nhu cầu cụ thể phụ thuộc tutorial được chọn.

C: Let's Build a Simple Database ↗

Python: ugit: Learn Git Internals by Building Git Yourself ↗

Web: server & browser

Đọc phần này trên GitHub Radar ↗ · Tài liệu gốc ↗

Web Server và Web Browser là hai góc nhìn khác nhau: một bên phục vụ nội dung, một bên tiếp nhận và trình bày nội dung. Dùng chúng để đặt câu hỏi về đường đi của một tương tác web. [S1]

Server: đi theo yêu cầu–phản hồi

Gợi ý học: xác định dữ liệu đầu vào, đoạn xử lý và phản hồi đầu ra. Bắt đầu bằng một yêu cầu đơn giản và một đường dẫn.

Browser: đi theo nội dung–hiển thị

Gợi ý học: quan sát cách một thành phần của trình duyệt biến nội dung thành kết quả nhìn thấy được. Kiểm tra tutorial xây phần nào, không suy ra nó hỗ trợ toàn bộ chuẩn web.

Đầu ra gợi ý

Một phản hồi hoặc ví dụ hiển thị nhỏ có thể thay đổi; kèm sơ đồ các bước và một tình huống chưa xử lý.

Điểm cần phân biệt

Tự viết server, dùng một framework để dựng API và tự xây browser engine là các bài học khác phạm vi. Đọc mục tiêu nguồn trước khi so sánh.

Python: Let's Build A Web Server. ↗

Python: Browser Engineering ↗

Ngôn ngữ & công cụ lập trình

Đọc phần này trên GitHub Radar ↗ · Tài liệu gốc ↗

Programming Language, Regex Engine, Shell và Template Engine cho bạn các hướng khám phá cách phần mềm đọc và xử lý ký hiệu hoặc lệnh. Chúng là những dự án khác nhau, không phải một bài tập chung. [S1]

Compiler và interpreter

Trình biên dịch và trình thông dịch là hai thuật ngữ thường gặp khi tìm hiểu ngôn ngữ. Hãy theo dõi đầu vào mã nguồn và cách tutorial tạo ra hành vi thực thi.

Regex, template và shell

Bạn có thể chọn một cơ chế hẹp để học: so khớp mẫu, thay chỗ trống trong nội dung, hoặc xử lý một dòng lệnh. Xác định đúng quy tắc mà phiên bản nhỏ hỗ trợ.

Đầu ra gợi ý

Một biểu thức, một mẫu hoặc một lệnh chạy được; một trường hợp không hợp lệ; và lời giải thích cách chương trình phân biệt chúng.

Cách tránh quá tải

Không thêm nhiều cú pháp hoặc tính năng trước khi tự giải thích được ví dụ đầu tiên. Khi mắc kẹt, tách phần đọc đầu vào khỏi phần thực thi.

JavaScript: The Super Tiny Compiler ↗

C: Tutorial - Write a Shell in C ↗

Hệ thống & hạ tầng

Đọc phần này trên GitHub Radar ↗ · Tài liệu gốc ↗

Docker, Operating System, Network Stack, Memory Allocator, Emulator và Distributed Systems mở ra các dự án về nền tảng vận hành. Tên nhóm rộng, nhưng từng hướng dẫn có phạm vi và điều kiện rất cụ thể. [S1]

Chọn một tầng để hiểu

Gợi ý: cơ chế cô lập, một bước giao tiếp mạng, cách cấp phát bộ nhớ hoặc một tập lệnh giả lập. Đừng gom mọi chủ đề hệ thống thành một dự án đầu tiên.

Đọc điều kiện môi trường

Kiểm tra hệ điều hành, kiến trúc máy, quyền chạy và công cụ mà tutorial yêu cầu. Một bài dành cho nền tảng này không tự áp dụng nguyên xi cho nền tảng khác.

Đầu ra gợi ý

Một cơ chế được minh họa bằng đầu vào nhỏ, kèm kết quả quan sát và sơ đồ tầng liên quan. Chỉ mở rộng sau khi hiểu phần đầu.

Giới hạn cần ghi

Bản học tập không tự có độ tin cậy, tính tương thích hoặc phạm vi như sản phẩm thật. Đánh giá việc sử dụng thực tế tách khỏi thành công của bài học.

Go: Building a container from scratch in Go ↗

Java: Building Your Own Kafka-like System From Scratch: A Step-by-Step Guide ↗

AI, neural network & nhận dạng

Đọc phần này trên GitHub Radar ↗ · Tài liệu gốc ↗

README có nhóm AI Model, Neural Network và Visual Recognition System. Có các liên kết về LLM, diffusion, tìm kiếm tài liệu và mạng neural; lựa chọn phụ thuộc vào điều bạn muốn hiểu và nền tảng đang có. [S1]

Đừng chọn chỉ vì nhãn AI

Viết rõ câu hỏi: muốn hiểu phép tính trong mạng nhỏ, quy trình xử lý văn bản hay một hệ thống tìm kiếm tài liệu? Những mục tiêu đó có đầu ra khác nhau.

Điều cần kiểm tra ở nguồn

Ngôn ngữ, toán nền, dữ liệu, thư viện và phần cứng được yêu cầu. Không suy ra rằng mọi bài "from scratch" đều không dùng thư viện.

Đầu ra gợi ý

Một ví dụ nhỏ có thể giải thích đầu vào, bước xử lý và đầu ra; ghi lại dữ liệu và điều kiện chạy để kết quả có thể được xem lại.

Chọn theo phạm vi

Nếu mới bắt đầu, ưu tiên bài có giả định và ví dụ cụ thể phù hợp kiến thức hiện tại. Report không xếp hạng độ khó chính thức của các tài liệu.

Python: A Large Language Model (LLM) ↗

Python: Implement a Neural Network from Scratch ↗

Đồ họa, game & mô phỏng

Đọc phần này trên GitHub Radar ↗ · Tài liệu gốc ↗

3D Renderer, Game, Physics Engine và Voxel Engine cho phép nhìn thấy kết quả của thuật toán. Kết quả trực quan là lợi thế, nhưng vẫn cần phân biệt phần cơ chế mà bạn đang thực sự tự xây. [S1]

Đặt câu hỏi theo hình ảnh

Ví dụ của người viết: một điểm trong mô hình trở thành điểm ảnh như thế nào, hoặc một thay đổi trong trạng thái làm vật thể chuyển động ra sao?

Chọn một sản phẩm nhỏ

Có thể là một cảnh đơn giản, một hình được dựng hoặc một luật tương tác. Hãy dùng mục tiêu mà tutorial thật sự hỗ trợ để chốt bài.

Đầu ra gợi ý

Lưu ảnh hoặc đoạn mô tả kết quả trước/sau một thay đổi. Dự đoán thay đổi trước khi chạy, rồi so sánh điều quan sát được.

Đừng chỉ nhìn hình đẹp

Thử giải thích dữ liệu và các bước biến đổi. Nếu chỉ chạy lại ví dụ mà không hiểu được một thay đổi nhỏ, quay lại phần cơ chế.

C++: Ray Tracing in One Weekend ↗

C++: How OpenGL works: software rendering in 500 lines of code ↗

Giao diện & công cụ cá nhân

Đọc phần này trên GitHub Radar ↗ · Tài liệu gốc ↗

Những nhóm như Front-end Framework / Library, Text Editor, Command-Line Tool, Bot và Template Engine gắn việc học với công cụ dễ hình dung. Bạn có thể chọn bài gần công việc hằng ngày để có câu hỏi cụ thể hơn. [S1]

Dùng trải nghiệm sẵn có

Nếu thường dùng framework giao diện, hãy hỏi nó biến trạng thái thành kết quả hiển thị thế nào. Nếu dùng công cụ dòng lệnh, hãy hỏi cách nó nhận và xử lý tham số.

Giới hạn một hành vi

Ví dụ của người viết: chỉ xử lý một loại đầu vào, một màn hình hoặc một thao tác sửa văn bản. Đừng đặt mục tiêu thay thế toàn bộ công cụ đang dùng.

Đầu ra gợi ý

Một tác vụ nhỏ có thể làm lại và một thay đổi do chính bạn bổ sung. Ghi điều kiện hoạt động cùng những trường hợp chưa hỗ trợ.

Đọc tiếp khi cần

Các nhóm còn lại, như BitTorrent, AR hoặc blockchain, vẫn có trong bản đồ. Hãy dùng cùng phương pháp chọn phạm vi và kiểm tra điều kiện đầu vào.

JavaScript: A DIY guide to build your own React ↗

C: Build Your Own Text Editor ↗

Ví dụ thật: bước đầu làm database

Đọc phần này trên GitHub Radar ↗ · Tài liệu gốc ↗

Phần 1 của cstack/db_tutorial chưa xây một cơ sở dữ liệu hoàn chỉnh. Nó bắt đầu bằng REPL: lặp việc đọc đầu vào, xử lý và in kết quả. Đây là ví dụ rõ về cách một dự án lớn có thể bắt đầu từ hành vi rất nhỏ. [S2]

Nguồn thực sự làm gì?

Chương trình in “db > ”, đọc một dòng; nhận .exit thì giải phóng bộ nhớ và thoát. Lệnh chưa nhận diện được sẽ được báo lại rồi tiếp tục vòng lặp.

Một kiểm tra được nguồn minh họa

Mở chương trình bằng ./db, nhập .tables: chương trình báo Unrecognized command '.tables'. Nhập .exit: chương trình kết thúc. Đây là hành vi được minh họa trong tài liệu, không phải SQL đã chạy được.

Điều kiện cần xem trước

Bài dùng C, cấu trúc InputBuffer và getline(). Kiểm tra môi trường của bạn theo nguồn; phần này chưa nhằm hỗ trợ nhiều nền tảng.

Tự kiểm tra điều đã hiểu

Bạn có giải thích được vì sao cần vòng lặp, ký tự xuống dòng được xử lý ở đâu và bộ nhớ được giải phóng khi nào? Chỉ học phần tiếp theo sau khi nắm những điểm này.

Cstack — Part 1: Introduction and Setting up the REPL ↗

Trang tutorial được BYOX giới thiệu ↗

Lộ trình gợi ý theo nền tảng

Đọc phần này trên GitHub Radar ↗ · Tài liệu gốc ↗

Các lộ trình dưới đây do người viết đề xuất để giảm việc phải học nhiều thứ mới cùng lúc. Chúng không phải thứ tự bắt buộc hoặc xếp hạng độ khó do repo công bố. [S1]

Bạn đã quen Python

Xem các lựa chọn Python ở Web Server, Git hoặc Database. Chọn một tutorial có yêu cầu khớp với điều bạn đã biết, rồi hoàn thành một mốc nhỏ.

Bạn đã quen JavaScript

Khám phá Template Engine, Front-end Framework / Library hoặc một compiler nhỏ bằng JavaScript. Đặt mục tiêu giải thích được một cơ chế.

Bạn đã học C/C++ hoặc hệ thống

Cân nhắc Text Editor, Shell, Memory Allocator hoặc bài Database bằng C. Xác minh yêu cầu môi trường của từng nguồn trước.

Bạn chưa tự viết chương trình nhỏ

Bổ sung nền tảng ngôn ngữ, hàm, vòng lặp và cách chạy chương trình trước. Sau đó chọn bài hẹp, có đầu vào/đầu ra rõ, thay vì bắt đầu bằng OS hay AI model lớn.

Tự kiểm tra: hiểu hay mới làm theo?

Đọc phần này trên GitHub Radar ↗ · Tài liệu gốc ↗

Một chương trình chạy được là tín hiệu tốt, nhưng chưa đủ để biết bạn đã hiểu cơ chế. Dùng các câu hỏi dưới đây để tự đánh giá bài thực hành. Đây là rubric học tập của người viết. [S1]

1 / Giải thích

Mô tả đầu vào → các bước xử lý → đầu ra bằng lời của bạn. Nêu rõ phần mã hoặc dữ liệu tương ứng trong phiên bản đã xây.

2 / Dự đoán

Trước khi thay một đầu vào, hãy viết điều bạn mong đợi. So sánh dự đoán với kết quả và giải thích khi chúng khác nhau.

3 / Thay đổi

Bổ sung một hành vi nhỏ mà không chép nguyên lời giải. Nếu mắc kẹt, xác định chính xác điều chưa hiểu và đọc lại phần liên quan.

4 / Biết giới hạn

Nêu ít nhất một tình huống phiên bản chưa xử lý. Tách rõ tính năng đã làm, tính năng mới đọc và tính năng chưa thử.

Những điều dễ hiểu nhầm

Đọc phần này trên GitHub Radar ↗ · Tài liệu gốc ↗

Tuyển tập hướng dẫn mở ra nhiều cách học, nhưng các tài liệu đích có tuổi đời, phạm vi và điều kiện riêng. Đọc đúng vai trò của repo sẽ giúp bạn đặt kỳ vọng phù hợp. [S1]

Clone repo ≠ tải mọi khóa học

Clone lấy nội dung của repo tổng. Bài viết, sách và mã được liên kết có thể nằm ở những nơi khác.

From scratch ≠ không dùng gì có sẵn

Mỗi nguồn chọn một mức nền tảng khác nhau. Đọc giả định, thư viện và thành phần mà bài yêu cầu thay vì suy ra từ tiêu đề.

Tất cả liên kết ≠ cùng điều kiện truy cập

Hãy kiểm tra nguồn còn truy cập được không, có yêu cầu tài khoản hoặc phần nội dung trả phí không. Report không xác nhận toàn bộ liên kết đều miễn phí hoặc còn hoạt động.

Bản học tập ≠ sản phẩm sẵn sàng triển khai

Giấy phép, tính tương thích, xử lý lỗi và độ đầy đủ tính năng cần được đánh giá riêng nếu muốn dùng trong dự án thật.

Phiếu bắt đầu dự án của bạn

Đọc phần này trên GitHub Radar ↗ · Tài liệu gốc ↗

Dùng phiếu này để chuyển từ đọc tổng quan sang một lần thực hành có mục tiêu. Có bản Markdown đi kèm trong repo để bạn tự điền. Các lựa chọn dưới đây là gợi ý của người viết.

Câu hỏi tôi muốn trả lời

Viết một câu: “Tôi muốn hiểu … bằng cách tự xây …”. Chọn đúng một nhóm và một hướng dẫn chính để bắt đầu.

Nguồn và điều kiện đầu vào

Ghi URL gốc, ngôn ngữ, môi trường, kiến thức đang có và điều cần bổ sung. Chưa xác minh được mục nào thì ghi rõ.

Phiên bản nhỏ nhất

Chốt một hành vi có đầu vào/đầu ra quan sát được. Ghi một mốc kiểm tra và một tình huống lỗi cần xem.

Điều kiện hoàn thành

Tôi chạy được mốc nhỏ, tự thay đổi được một phần, giải thích được cơ chế và biết giới hạn. Sau đó mới quyết định mở rộng hay chuyển chủ đề.

Nguồn & phạm vi kiểm chứng

Số nhóm và các liên kết được đối chiếu với README ở commit ghim. Ví dụ database dựa trên nội dung thật của Part 1. [S1][S2]

S1 / CodeCrafters — build-your-own-x / README

https://github.com/codecrafters-io/build-your-own-x/blob/aa17439b62f384511a5561ce308e9598b94d8989/README.md

S2 / Cstack — db_tutorial / Part 1

https://github.com/cstack/db_tutorial/blob/60b50c5b7be787a4aaa1e50ab8a90c6cabb75159/_parts/part1.md

Chưa chạy thử toàn bộ tutorial hoặc xác minh mọi liên kết đích còn hoạt động. Các mục tiêu, nhóm diễn giải, lộ trình và tiêu chí tự kiểm tra trong report là gợi ý của người viết. Giấy phép của từng tài nguyên cần được đọc tại nguồn tương ứng.